



University of Piraeus

School of Information and Telecommunications Technologies

Department of Digital Systems

Level: Postgraduate Program of Study

Advanced Cybersecurity and Artificial Intelligence Topics

Risk-Adaptive Access Control with Synthetic Enterprise Logs

Supervisors: Christos Xenakis, Apostolis Zarras, and Georgios Vouros

Kalaitzidis Ioannis

ioannis_kalaitzidis@ssl-unipi.gr

Piraeus

07/07/2026

Introduction

Each organisation bases its operation on controlling who has access to which resource and under what conditions. In modern environments, where users connect from everywhere and attackers move at machine speeds, the traditional model of the trusted internal network has collapsed. Its place is taken by the Zero Trust philosophy, which treats every request as potentially hostile and mandates continuous, contextual verification. Access control ceases to be a static binary decision and turns into a data problem: the slightest malicious activity must be distinguished within a huge volume of normal traffic, facing the classic challenges of volume, speed and noise (lecture 1).

The object of the thesis is not simply to demonstrate that a system can detect suspicious accesses, but to systematically measure how well it does it, in what way and with what limits. For this purpose, a controlled synthetic corporate environment was designed, realistic legal and malicious activity was produced, two different risk assessment technologies were applied and then combined into a single hybrid system.

There are two large families of technicians, and the whole work rests on their difference. The first is the rule-based approach. It is based on predetermined weights and thresholds of the form, if a specific high-risk pattern is detected, an alarm is sounded. It is transparent and accurate in obvious scenarios, but it has a fundamental weakness that theory also emphasizes (lecture 4): it does not perceive what depends on the context or history of a user. The second family is anomaly or machine learning-based detection. Instead of explicit rules, the system learns from examples what normal behavior looks like and means what deviates. Its advantage is that it catches complex, low-and-slow attacks. Its disadvantage is that it can also strike for unusual but legitimate behaviors, creating alert fatigue (alert fatigue, lecture 1).

Neither part is sufficient on its own. The solution is for them to cooperate, with clear rules for cases where they disagree. This is exactly the hybrid system built in this work: an unsupervised anomaly detector (Isolation Forest) powers a supervised classifier (Gradient Boosting), and underneath them deterministic Zero Trust rules act as a safety net.

Particular importance was given to the honesty of the evaluation. If a model is evaluated by leaking information from the future to the past (temporal leakage), it produces impressive but fake metrics. To avoid this, the events were classified chronologically and the separation of training and evaluation was done based on time, while all temporal characteristics were calculated causally. Thanks to this, the metrics reported are realistic (for example, the F1-score of the hybrid model amounts to 0.779 and the PR-AUC to 0.942) and reflect actual detection capability. This attitude is linked to the principles of responsible artificial intelligence (lecture 7). Three were observed:

- **Explainability:** the system explains why it made each decision, giving a degree of risk and a clear etiology (decision_reason) to each event, rather than acting as an opaque black box.
- **Accountability:** every decision is accompanied by its source, i.e. which part (the model or the deterministic rule) caused it, so that it can be controlled.
- **Honest report:** system failures such as incorrect alarms or performance crash under drift conditions were not hidden.

A corporate organization with 60 users, 4 departments and 30 resources of different sensitivity was chosen as the scope of application, which accepts both legitimate daily activity and six distinct types of attacks. The project answers seven questions. The corporate scenario will be designed (Task 1), differentiated anomalies will be injected (Task 2), a static rule engine will be implemented as a baseline (Task 3), the hybrid AI model will be trained (Task 4), risk will be transformed into dynamic decisions (Task 5), the system will be evaluated with explainability (Task 6) and its slippage and governance will be studied (Task 7). An optional fairness analysis task was also implemented. The following sections explain each of the above in detail.

Structure of the implementation

The system is not a single program, but a chain of four distinct scripts in Python, each with a single job. The 01_generate_dataset.py produces the movement, the 02_baseline_policy.py Baseline, the 03_ai_pipeline.py does the feature engineering, trains the hybrid model and generates the decisions, and the 04_drift_fairness_plots.py It outputs the assessment, fairness analysis, drift simulation, and graphs. The entire stream is fully reproducible through a fixed random seed (value 25012), so that each reference number is derived from the same, uniform execution of the Code.

Task 1: Design of a synthetic corporate scenario

Before any system is evaluated, a controlled environment is needed in which it is known with certainty what happened at any given moment. A 14-day synthetic corporate environment was created, starting on May 1, 2026, which includes **60 users**, **4 sections**, **4 roles**, **30 Resources** and overall **15,000 events** access. These figures exceed the minimum thresholds of pronunciation (50 users, 25 resources, 4 segments, 12,000 events, 14 days).

Identities, Roles, and Permissions (RBAC)

The 60 users (USR_001 to USR_060) were divided into the Engineering, Finance, HR and Sales departments, with the following distribution of roles:

Role	Percentage	Description
Intern	20%	Limited permissions, no access to privileged resources.
Employee	50%	Typical employee with access to department resources.
Manager	20%	Expanded access within the department.
Admin	10%	Increased management rights.

Each user has a personal behavioural baseline: a typical work start time, a preferred network zone allocation (Sales users rely more on VPN_Remote), and a small set of common resources. Modeling individual behavior is critical because it allows the next stage to measure the deviation of each event from the norm of the specific user and not just from the general norm of the segment.

Enterprise Resources and Data Schema

30 pores were defined with four levels of sensitivity. Confidential and Privileged resources are tightly linked to specific departments (for example, RES_FIN_BANK belongs to Finance and RES_ENG_PROD_DB to Engineering), which allows for the detection of horizontal movement between departments. Each record in the dataset includes the following fields:

Field	Description
event_id	Unique event identifier (time order).
timestamp	Second accuracy time stamp.
user_id, role, department	Identity and organizational details of the user.
network_zone	Zone of origin: Internal_Corporate, VPN_Remote, Public_Internet.
device_id	Device ID (one trusted per user or DEV_UNKNOWN).
resource_id, sensitivity	Target resource and sensitivity level.
action	Read, Write, Execute, or Delete.
outcome	Success or Failure.
is_anomaly, anomaly_type	Ground truth tag and anomaly category.

Why do the results come out the same every time?

Motion generation and model training use a fixed randomness number (seed 25012) so that each run gives exactly the same results. In addition, all analysis is done offline on a single data file, rather than relying on live production that would change each time.

Files implementing Task 1: The program **01_generate_dataset.py** designs the topology (users, roles, departments, resources) and produces the **data/synthetic_enterprise_logs.csv**, while the documentation file **SCHEMA.md** describes in detail the data schema and methodology.

Task 2: Injecting differentiated anomalies

6% of the incidents (900 out of 15,000) were deliberately modified to represent six distinct attack types, evenly distributed across 150 records per type. This percentage is within the recommended range (3 to 10%) and reflects the real base-rate problem of cybersecurity, where attacks are rare compared to legitimate traffic (lecture 5). The selection of six categories (instead of the five defined by the minimum) was deliberately made so that the dataset covers both simple, static-detectable anomalies and complex, contextual anomalies.

Anomaly Type	Construction Logic	Security Question
Impossible_Travel	Successful access from Public_Internet.	Is a perimeter violation detected?
Off_Hours_Privileged	Access to privileged resources 01:00-04:00.	Is high-risk access detected outside working hours?
New_Device_Sensitive	Unknown device with Delete on sensitive data.	Is dangerous activity detected by an untrusted endpoint?
Unusual_Resource_Access	Access to another department's resource, within working hours.	Is lateral movement or RBAC violation detected?
Bulk_Data_Spike	Burst of lightning-fast Read actions from the same user.	Is there a pattern of data exfiltration?
Privilege_Escalation	Failed connections followed by success.	Is brute-force or credential stuffing detected?

The last three categories (Unusual_Resource_Access, Bulk_Data_Spike, Privilege_Escalation) are deliberately context-dependent. No single field value is suspicious. The anomaly becomes visible only when the event is examined in relation to the user's history, the frequency of requests, or the behavior of their team. It is precisely these categories that highlight the limit of static rules and justify the transition to behavioral AI.

Class imbalance

The fact that only 6% of incidents are attacks is deliberate and realistic. This imbalance is important because it obliges the evaluation not to be based on general accuracy. A system that would always say there is no attack would be right 94% of the time, but without having caught a single attack. For this reason, more appropriate metrics are used (Precision, Recall, F1, PR-AUC), as explained in Task 6 (lecture 5).

Files implementing Task 2: the same program **01_generate_dataset.py** injects the six types of anomalies and records them in the `is_anomaly` and `anomaly_type` columns of the file **data/synthetic_enterprise_logs.csv** (The Ground Truth).

Task 3: Static Rules Machine (Baseline)

As a point of reference, a deterministic risk calculation engine was developed. It analyzes each event independently and adds up penalties based on predetermined weights, starting from a base value (Base Risk = 10) and limiting the final score to the range 0 to 100. The decision thresholds are: ALLOW for scores below 45, CHALLENGE for 45 to 74 (MFA requirement), and BLOCK for 75 and above.

Variable	Status	Weight	Justification
Network Zone	Public_Internet/VPN_Remote	+35/+15	Access outside the corporate perimeter.
Time-of-Day	Night (22:00-05:00)	+20	Suspicious activity outside working hours.
Sensitivity	Privileged/Confidential	+25/+15	Depending on the criticality of the resource.
Action	Delete/Write	+15/+5	Irreversible or amending actions.

Variable	Status	Weight	Justification
Device Trust	DEV_UNKNOWN	+30	Use of an unregistered device.
Outcome	Failure	+10	Failed authentication attempt.

The baseline assessment revealed a clear pattern: success in simple anomalies and complete failure in context-dependent. The detection index by type (detection = BLOCK or CHALLENGE in true anomaly) was:

Anomaly Type	ALLOW (escape)	Localization (%)
Impossible_Travel	0	100.0%
Off_Hours_Privileged	0	100.0%
New_Device_Sensitive	0	100.0%
Privilege_Escalation	0	100.0%
Bulk_Data_Spike	150	0.0%
Unusual_Resource_Access	150	0.0%

At the overall level, the baseline achieved Precision 0.447, Recall 0.667 and F1-score 0.535. The critical finding is that the categories Bulk_Data_Spike and Unusual_Resource_Access escape in their entirety (100% escape). The reason is structural: the machine evaluates each log as a single unit (stateless), with no time correlation or knowledge of regularity per user. A single Read request to a legitimate resource, within working hours and from a trusted device, is considered perfectly normal, even when it is part of a burst of 25 requests in a matter of seconds. The conclusion is in line with the theory (lecture 4): static methods are not sufficient against behavioral threats and a transition to UEBA models is required.

Files implementing Task 3: The program **02_baseline_policy.py** applies the deterministic rules to each event and produces the file **data/baseline_decisions.csv**, which contains the baseline_risk_score and the baseline_decision (ALLOW/CHALLENGE/BLOCK).

Task 4: Hybrid AI Risk-Scoring Model

The approach is hybrid on two axes. First, it combines an unsupervised anomaly detector with a supervised classifier. Second, it places deterministic Zero Trust rules as a safety net under the model. This option substantially differentiates the implementation from a simple use of a single classifier.

- **Unsupervised signal (Isolation Forest):** trained on the behavioural features of the training window. Isolates anomalies with random trees, where anomalies are separated into fewer steps (lecture 3). The anomaly score it produces is reintroduced as an additional trait.
- **Supervised scorer (Gradient Boosting):** a HistGradientBoostingClassifier consumes all attributes and produces a calibrated probability, scaled to 0 to 100. Boosting in tabular attributes is highly efficient (lecture 3).

The problem remains anomaly detection: anomalies account for only 6% (class imbalance). To address the imbalance, sample weights were used that force the model to punish more severely the escape of an anomaly.

Prevent Time Slippage

In security logs, a random split by shuffle would allow the model to use information from the future to predict the past, producing fictitiously perfect metrics. To avoid this, the events are strictly chronologically classified and divided based on time: the first 70% is the training window and the last 30% (4,500 events, of which 248 are anomalies) is the held-out evaluation window. In addition, all rolling features are calculated causally, taking into account only the past, reproducing the operation of the system in real time.

Feature engineering

For each event, eleven traits were extracted that combine behavioral and contextual signals. The framework they offer is exactly what static rules lack:

Feature	Definition and purpose
access_frequency	User requests in the previous 5 minutes (detects Bulk_Data_Spike).

recent_failure_count	User failures in the previous 10 minutes (detects Privilege_Escalation).
session_resource_variety	Distinct resources of the user in the previous 15 minutes.
is_off_hours	Access index at suspicious hours (22:00-05:00).
device_novelty	First appearance of the user on this device or unknown device.
sensitivity_mismatch	Low-role user touching Confidential/Privileged data.
peer_rarity_score	Scarcity of the resource for the user's part (running).
user_hour_deviation	Deviation of the time from the user's average access time.
zone_risk	Numerical encoding of the network band (0/1/2).
cross_department	Access to a resource of another department (lateral movement signal).
iso_score	Anomaly score of Isolation Forest (hybrid unsupervised signal).

Files implementing Task 4: The program **03_ai_pipeline.py** calculates the eleven traits, trains the Isolation Forest and the HistGradientBoosting classifier with temporal split, and produces the **data/evaluated_risk_logs.csv** (full log with all features, the calculated_risk_score_ai and the final decision).

Task 5: Dynamic Decision Engine

The decision engine converts the AI risk score into a final decision, while applying deterministic Zero Trust rules as contextual overrides. The logic is hierarchical:

- **Critical risk (score ≥ 75):** immediate BLOCK and SOC alert. Corresponds to a high probability of active compromise.
- **External network safety net:** any successful access by Public_Internet is not automatically approved, regardless of score, but is at least led to a CHALLENGE (perimeter breach guarantee).
- **Medium risk ($45 \leq \text{score} < 75$):** CHALLENGE with step-up MFA. The etiology is specified according to the dominant signal (new device, off-hours, repeated failures).
- **Low risk with lateral movement:** even with a low score, if sensitivity_mismatch and cross_department coexist, the decision is upgraded to CHALLENGE with manager approval. It implements the principle of never trust, always verify.

- **Safe baseline:** immediate ALLOW, as the activity is identical to the normal behavior of the user.

The generated file constitutes the Dynamic Access Audit Log. In the Zero Trust architecture, it represents the output of the Policy Decision Point (PDP) sent to the Policy Enforcement Point (PEP). The distribution of decisions across the 15,000 incidents was 13,547 ALLOW (90.3%), 1,070 BLOCK (7.1%) and 383 CHALLENGE (2.6%). The dominance of ALLOW proves that the system does not cause operational suffocation, leaving the vast majority of legitimate users to work frictionlessly.

Files implementing Task 5: The decision-making machine is implemented within the `03_ai_pipeline.py` and produces the `data/dynamic_access_decisions.csv`, the marker-facing file with the fields `event_id`, `calculated_risk_score_ai`, `final_decision` and `decision_reason`.

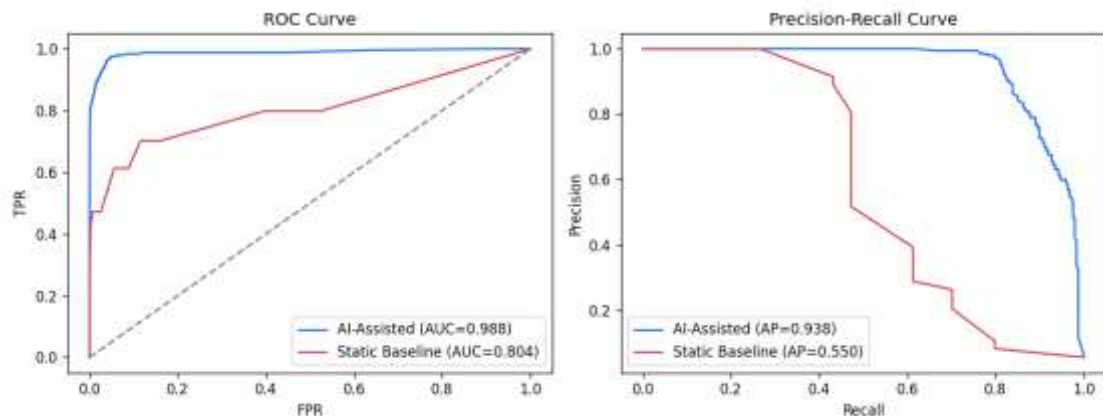
Task 6: Evaluation and explainability

The evaluation was carried out in the held-out evaluation window (last 30% in time), which did not participate in the training at all. The table compares the AI-Assisted system with the static baseline in exactly the same set, for the anomaly class (Class 1):

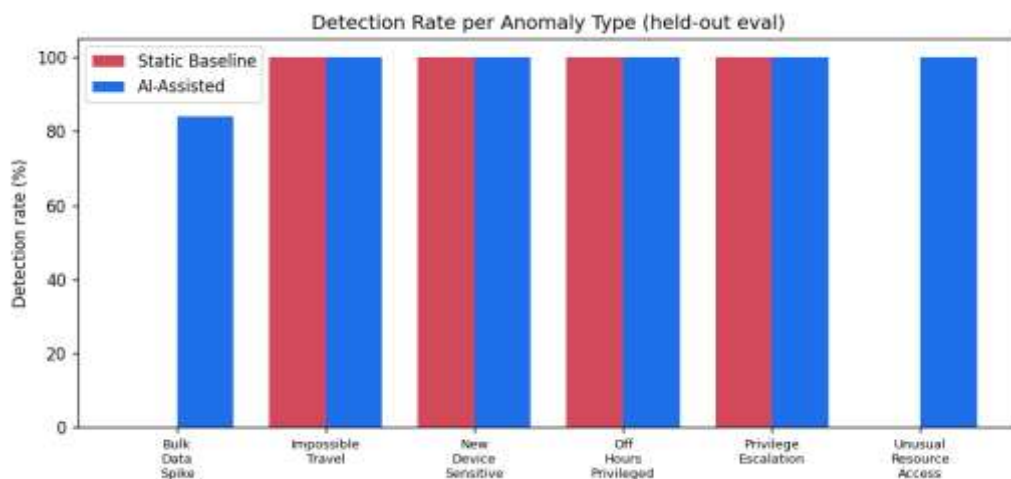
Metric (Class 1)	Static Baseline	AI-Assisted	Operational Importance
Precision	0.392	0.677	Reduction of false alarms (alert fatigue).
Recall	0.613	0.919	Increased detection of complex threats.
F1-score	0.478	0.779	Better balance of security and usability.
ROC-AUC	0.804	0.989	Superior separation capability overall.
PR-AUC	0.550	0.942	Resistance to class imbalance.

The numbers are for the AI model. When the full decision engine with Zero-Trust overrides is implemented, Recall goes up to 0.968 with Precision at 0.580. The slight reduction in Precision is by design acceptable: it is due to the external network's safety net, which leads to CHALLENGE (and

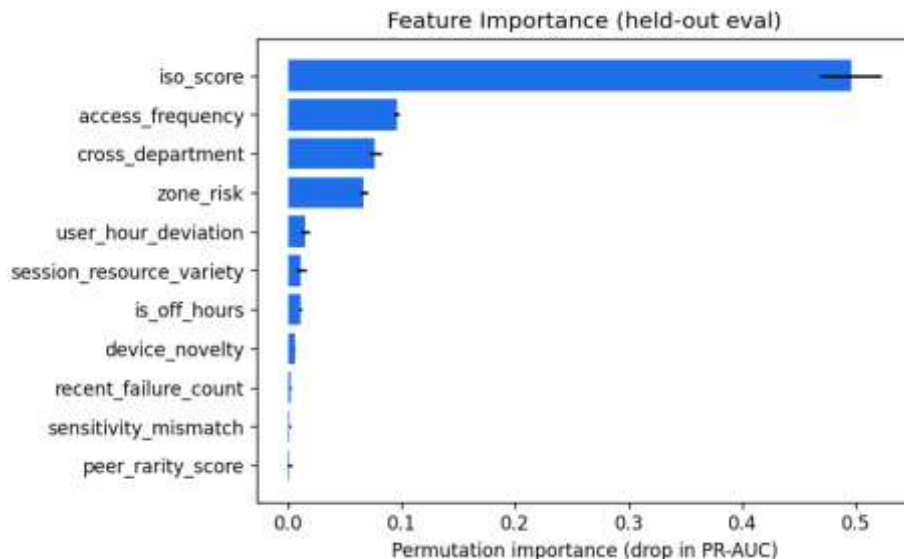
not BLOCK) some legitimate accesses by Public_Internet. A CHALLENGE is not a lockout but a simple MFA requirement, so the operational costs are minimal.



The essential superiority of AI can be traced back to Recall. The model catches 92% of the anomalies versus 61% of the baseline. Because attacks are rare, emphasis was placed on the Precision-Recall curve (lecture 5), where the AI-Assisted system reaches a PR-AUC of 0.942 versus 0.550 of the baseline.



The above figure captures the central argument of the work. The baseline detects 0% of Bulk_Data_Spike and Unusual_Resource_Access. The AI-Assisted system raises them to 84% and 100% respectively, maintaining 100% in the rest of the categories. The detection of Bulk_Data_Spike is due to access_frequency, while Unusual_Resource_Access is due to cross_department and iso_score.



The significance analysis (figure above) confirms the logic of hybrid design. Isolation Forest's `iso_score` is the strongest trait, followed by `access_frequency`, `cross_department`, and `zone_risk`. That is, the behavioural and contextual signals, which are completely absent from the baseline, are what determine the superior performance.

Event-level explainability

To ensure responsible use (lecture 7), each decision is accompanied by a clear statement of reasons. Here are five real-life examples from the Audit Log, one per anomaly type:

- **EVT_10521 (Unusual_Resource_Access, 100/100, BLOCK):** `USR_028` (Employee, Engineering) requested Read in the HR department `RES_HR_RECORDS` (`sensitivity_mismatch` = 1, `cross_department` = 1). Although within working hours and from a trusted device, it was recognized as lateral movement.
- **EVT_13863 (Bulk_Data_Spike, 100/100, BLOCK):** `USR_014` executed 24 Read requests in a matter of seconds (`access_frequency` = 24). Each individual request was legitimate, but the frequency photographs data exfiltration.
- **EVT_10752 (New_Device_Sensitive, 100/100, BLOCK):** Delete action on the privileged `RES_FIN_BANK` from an unknown device (`DEV_UNKNOWN_1957`). The convergence of `device_novelty`, destructive energy, and high sensitivity led to blocking.
- **EVT_11650 (Privilege_Escalation, 100/100, BLOCK):** the `USR_017` experienced repeated authentication failures by `Public_Internet`, followed by success. The pattern corresponds to successful brute-force or credential stuffing.

- **EVT_12831 (Impossible_Travel, 94/100, BLOCK):** successful access by Public_Internet. The model yielded a high score, while the deterministic net guarantees that no such access is implicitly approved.

Operational risk analysis and by groups

The cost of False Positives is addressed through the CHALLENGE status, which absorbs legitimate users with unusual traffic with a simple MFA requirement instead of a logout. The cost of False Negatives is addressed through deterministic overrides, which stop access to a sensitive off-segment resource even with a low score. Performance was also broken down by sensitivity level:

Sensitivity	Events	Anomalies	Precision	Recall	F1	FPR
Public	611	31	0.528	0.903	0.667	0.043
Internal	3196	54	0.280	0.907	0.428	0.040
Confidential	429	64	0.901	1.000	0.948	0.019
Privileged	264	99	0.925	1.000	0.961	0.048

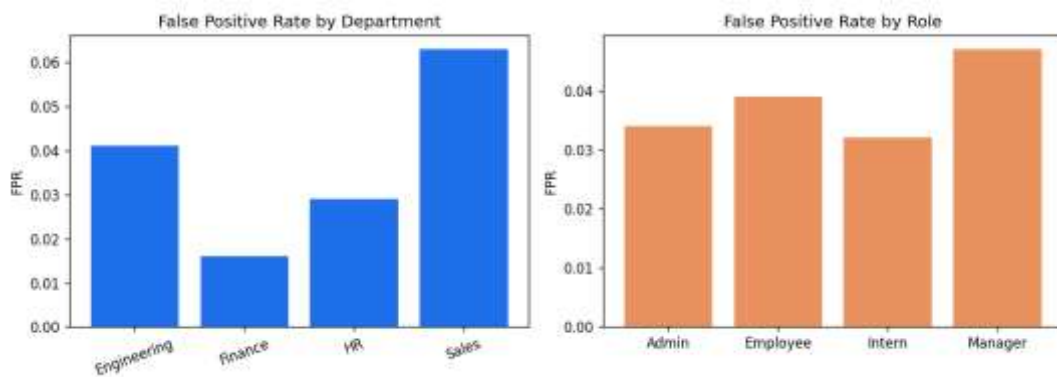
The system is visibly stronger in critical resources (Confidential F1 0.948, Privileged F1 0.961), i.e. exactly where the risk is greater. The lower Precision in Internal resources is due to the small anomaly base there, where even a few False Positives disproportionately affect the metric.

Files that implement Task 6: The program **04_drift_fairness_plots.py** produces the rating charts (**figures/fig1.. fig4**) and calculates the breakdown per group (segment, role, sensitivity) in the files **data/fairness_by_*.csv**. The main metrics of the model (Precision, Recall, F1, ROC-AUC, PR-AUC) are calculated in **03_ai_pipeline.py**.

Optional Task: Fairness Analysis

As part of the optional advanced work, the assessment was extended to a fairness analysis across roles and departments, in line with the pillars of Responsible AI (lecture 7): privacy, accountability, explainability, and trust. The aim is to identify any disproportionate false positive (FPR) rates per group, which would mean that some employees are systematically suffering more.

Role	Events	Anomalies	Precision	Recall	F1	FPR
Admin	436	28	0.667	1.000	0.800	0.034
Employee	2391	109	0.551	1.000	0.710	0.039
Intern	738	77	0.767	0.896	0.826	0.032
Manager	935	34	0.447	1.000	0.618	0.047
Department	Events	Anomalies	Precision	Recall	F1	FPR
Engineering	1433	50	0.467	1.000	0.637	0.041
Finance	922	104	0.881	0.923	0.901	0.016
HR	1015	37	0.569	1.000	0.725	0.029
Sales	1130	57	0.456	1.000	0.626	0.063



The analysis revealed mild but real differences. The Sales department and the Manager role show the highest FPR (0.063 and 0.047). The interpretation makes sense: Sales users rely more on VPN_Remote and non-standard patterns, which the model tends to interpret as slightly more suspicious. In contrast, segments with more predictable behavior (Finance, FPR 0.016) exhibit lower percentages.

Suggestion for improvement: the introduction of a separate behavioural baseline per department or per role would reduce these discrepancies, directly linking to the pillar of trust (fairness) of Responsible AI.

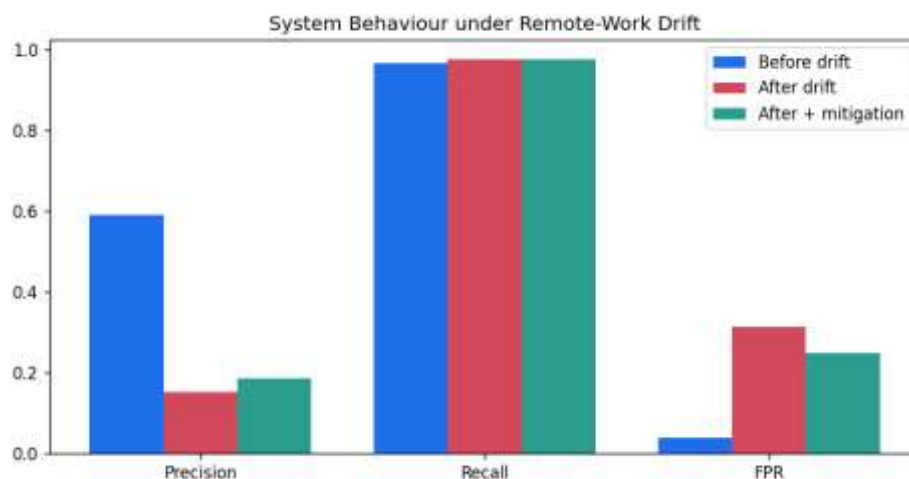
Files that implement the optional Task: The fairness analysis is implemented in the `04_drift_fairness_plots.py` and is reflected in the archives `data/fairness_by_department.csv`, `fairness_by_role.csv` and in `fairness_by_sensitivity.csv`.

Task 7: Skidding simulation and governance

A Zero Trust system based on Machine Learning cannot be evaluated on a static data snapshot alone. To assess its resilience, the system underwent a stress-test through a simulation of a realistic environment change scenario (concept drift): a week of universal teleworking. The following changes were applied to the evaluation window, without any modification to the trained model:

- All connections Internal_Corporate moved to VPN_Remote (geographical dispersion).
- Schedules shifted, increasing accesses at suspicious hours (is_off_hours).
- 25% of devices were marked as new (an increase device_novelty due to personal devices).

Status	Precision	Recall	FPR	ROC-AUC
Before drift (baseline operation)	0.580	0.968	0.039	0.989
After the drift (remote week)	0.152	0.976	0.317	0.927
After + mitigation (VPN -30)	0.180	0.976	0.260	0.930



The change was rapid. While Recall remained stable, Precision collapsed from 0.580 to **0.152**, with FPR skyrocketing from 3.9% to 31.4%. The model, having been trained on the corporate baseline, interpreted the massive change of zones and hours as a generalized attack, causing a barrage of false alarms and operational suffocation.

Response Strategies and Governance

- **Rule Updates:** temporary rule that deducts 30 points from each connection via a corporate VPN. It reduces FPR from 31.4% to 24.9%, but does not fully reset it, proving that rule-update alone is not enough.
- **Threshold Recalibration:** temporary movement of the ALLOW threshold to absorb the justified behavior change.
- **Model Retraining (long-term solution):** waiting 7 to 14 days for new data collection and retraining so that teleworking can be integrated into the new baseline.
- **Governance Review:** no changes to thresholds or models are automatically applied. Approval by the organization's Change Advisory Board (CAB) is required.

Policy Drift Risk

If the model is allowed to retrain completely automatically, a slow and steady exfiltration attack (low-and-slow) can be gradually incorporated as the new normal baseline. This requires human-in-the-loop approval of high-impact tasks (lecture 6), model versioning for full auditability, and solid explainable contextual rules so that, even if the model fails due to drift, the final decision remains understandable and human-controlled.

Files implementing Task 7: The program `04_drift_fairness_plots.py` simulates the teleworking scenario, re-evaluates the system and produces the `data/drift_summary.csv`.

Execution conditions and parameters

In order for anyone to reproduce the exact same results, the execution conditions are recorded:

- **Data:** the analysis is done offline on a single synthetic set of 15,000 events, without live production.
- **Repeatability:** constant seed of randomness (25012) in data generation and training, so that each execution gives the same results.
- **Models:** Isolation Forest (200 trees, contamination 0.06) and HistGradientBoostingClassifier (300 iterations, max_depth 6), from the scikit-learn library.

- **Environment:** Windows with Python 3.10 and pandas, numpy, scikit-learn, matplotlib libraries (directory in requirements.txt). The first two stages also run on Pure-Python for compatibility with Application Control.

The entire chain runs by executing the four scripts sequentially (01 up to 04), as documented in the README.md.

Limitations and Threats to Validity

In the spirit of honest reporting (lecture 7), the limits within which the conclusions apply are explicitly stated:

- **Synthetic data:** the results are derived from a controlled, synthetically produced activity. It is not documented that the performance is transferred as such to a production network with a different distribution.
- **Range of attacks:** metrics are for six specific types of anomalies and are not generalized to truly unknown (zero-day) threats.
- **Parameter selection:** baseline weights, decision thresholds, and superparameters of the models were empirically selected and would need to be reconfigured in another environment.
- **Susceptibility to drift:** as Task 7 showed, performance is not stable over time and requires active governance and periodic retraining.

These limitations do not negate the central finding, but rather precisely delineate its scope and indicate specific directions for future research.

Conclusion

A comprehensive and repeatable Zero-Trust Adaptive Access Control framework was constructed, which combines a static rule mechanism with a hybrid Machine Learning model. The work did not stop at demonstrating detections, but treated the problem as something measurable, with a full cycle, from the design of the scenario and the injection of anomalies to dynamic policy enforcement and systematic evaluation.

As for the weakness of individual approaches, the static baseline showed complete blindness (0% localization) in context-dependent attacks Bulk_Data_Spike and Unusual_Resource_Access. The hybrid model (Isolation Forest and Gradient Boosting) covered exactly these blind spots, raising

the F1-score from 0.478 to 0.779 and the PR-AUC from 0.550 to 0.942. Most importantly, the numbers did not come out artificially perfect, precisely because temporal leakage was avoided.

Three lessons summarize the work. Firstly, temporal correlation and per-user behavioral analysis are essential for detecting complex threats. Secondly, AI is not enough on its own: deterministic Zero-Trust overrides act as a necessary safety net. Third, the drift test proved that performance is not stable and that governance, controllability, and human control are critical to the sustainable operation of an ML system in cybersecurity. The hybrid architecture offers the ideal balance: it introduces dynamic risk consideration without sacrificing business continuity and organizational stability.

Deliverables and execution instructions

Submission is organized in a single folder, with a clear separation between code, data, and documentation. The four scripts are located at the root of the folder, the generated data in the data/ folder and the graphs in the figures/. Full instructions can be found in README.md and SCHEMA.md. The following table shows which file implements each request:

Task	Implemented by
1. Scenario Planning	01_generate_dataset.py, SCHEMA.md, data/synthetic_enterprise_logs.csv
2. Injection of abnormalities	01_generate_dataset.py, data/synthetic_enterprise_logs.csv (labels)
3. Static Rule Machine	02_baseline_policy.py, data/baseline_decisions.csv
4. AI risk-scoring model	03_ai_pipeline.py, data/evaluated_risk_logs.csv
5. Dynamic Decision Engine	03_ai_pipeline.py, data/dynamic_access_decisions.csv
6. Evaluation and explainability	04_drift_fairness_plots.py, figures/fig1.. fig4, data/fairness_*.csv
7. Glide and Governance	04_drift_fairness_plots.py, data/drift_summary.csv, figures/fig6_drift.png
Optional: Fairness analysis	04_drift_fairness_plots.py, data/fairness_by_*.csv, figures/fig5_fairness.png

In addition to the code, the submission includes this report, documentation files (README.md, SCHEMA.md) and the list of dependencies (requirements.txt).

